CS 360
Programming Languages
Day 30

Final Project

- **Schedule your checkpoint with me!**

Review

```
30 rem print 'ignored'
35
50 print "never gets here"
40 end
60 print 'three' : print 1.0 + 3
70 goto 11. + 18.5 + .5 rem ignored
10 print "o" ; "n" ; "e"
20 print : goto 60.0 : end
```

Review

- Let’s look at the the Basic lexer before moving on!
  - What are source locations? Why do we embed them in tokens?
  - Why is the lexer separated from the tokenizer now?
  - Why do we **skip** whitespace instead of calling **next-token**?

Installing Custom Packages

- From inside the comp360_sp26 directory:

```
> raco pkg install basic
```

- Notice we can now access modules via `basic/my-module`

- See **parse-test.rkt** and **parse-only-test.rkt**

Tokenizer

- Read **tokenizer.rkt**
- Ask a question!
- Notice how simple the tokenizer is when the lexer is separated

Parser: What tokens are available?

- NEWLINE
- REM
- INTEGER or DECIMAL
- STRING
- Statement names, like “print”, “goto”, “end”
- “+” (addition)
- “:” (multiple statements in one line)
- “;” (separates print items)

Parser

- Brainstorm some parser rules!
- Start here: a basic program is defined as …

Parser

- A basic program is some basic lines, separated by new lines
- A basic line is a LINE NUMBER, optionally followed by some stuff
- That “stuff” is a basic statement, which is either a single statement or several separated by “:”
- A line number is an integer
- A statement is either “end”, “print” or “goto”, but these each have their own grammar
- REM indicates a comment
- “print” is much like a statement, except that it can be separated by “;” instead of “:”; we can print a string or an expression
- “goto” requires an expression which evaluates to a number
- An expression is the sum of some numbers

Parser

- Read **parser.rkt**
- Ask a question!

Parser: Cuts and Splices

- What would happen without all those special characters in the parser?

- “/” in a parser rule **cuts** that symbol out of the parse tree: once we’ve parsed the source, we should toss out tokens that are really only for the parser (instead of the expander)

- “@” before a parser rule **splices** those rules into the defining rule, reducing nestedness / increasing readability in the parse tree

Testing

- Run **parser-test.rkt** and **lexer-test.rkt**
- Modify them for an additional test

Debugging Dialects

- Wouldn’t it be nice to run our source through a language specially designed to *only* parse the source?

- Consider a dialect: **basic/parse-only** which merely parses a source or **basic/tokenize-only** which merely tokenizes a source

- See **parse-only.rkt, tokenize-only.rkt** and their corresponding tests

Errors!

- Modify **parse-only-test.rkt** so that it has a lexing error.
- What happens? Why?

- Modify **parse-ony-test.rkt** so that it has a parsing error.
- What happens? Why?

- Say “Thank you!!!” to source locations!